UNIFECAF · ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Sistema Automotivo — Gestão de Estoque de Veículos

Parte Teórica — Relatório de Análise e Projeto

Aluno	Breno Silva de Abreu
Matrícula	189836
Disciplina	Object Oriented Programming
Tecnologias	Java 21 · Spring Boot 3.3 · MySQL 8
Repositório	https://github.com/PharmaBR/sistema-automotivo-unifecaf
Data	Outubro de 2026

Sumário

1. Contexto e problema
2. Levantamento de requisitos (entrevista simulada)
3. Entidades do sistema
4. Aplicação da Programação Orientada a Objetos
5. Requisitos funcionais e não funcionais
6. Justificativa das tecnologias

1. Contexto e problema

Concessionárias e lojas de veículos precisam saber, a todo momento, quais carros estão no pátio, por quanto estão anunciados e em que situação se encontram. Quando esse controle é feito em planilhas ou anotações espalhadas, as informações ficam desatualizadas: vendedores oferecem carros já vendidos, clientes não encontram o que procuram e a gerência não consegue acompanhar o giro do estoque.

Este projeto desenvolve um sistema de **gestão de estoque de veículos** que centraliza o cadastro de marcas, modelos e veículos, permite atualizar preço, quilometragem e situação, remover veículos que saíram do estoque e, principalmente, **consultar e filtrar** o estoque por características como marca, tipo, preço e ano.

Afetados pelo problema: vendedores (informação imprecisa no atendimento), clientes (dificuldade para encontrar o veículo desejado) e gerentes/administradores (sem visão consolidada para decidir).
Solução adotada: API REST em Java com Spring Boot, persistência em MySQL e uma interface web simples.

2. Levantamento de requisitos

O levantamento foi feito por meio de uma **entrevista simulada** com o gerente de uma concessionária fictícia (“AutoPrime Veículos”). Para cada pergunta, registra-se a resposta simulada e o requisito que ela originou.

1. Como o estoque de veículos é controlado hoje?

Resposta simulada: Em planilhas compartilhadas e anotações dos vendedores. Cada um atualiza quando lembra, então é comum a planilha dizer que um carro está disponível quando ele já foi vendido.

→ Requisito derivado: Confirma a causa raiz (falta de organização) e a necessidade de uma fonte única de dados.

2. Quem vai usar o sistema e para quê?

Resposta simulada: Os vendedores, para consultar o estoque e reservar carros para clientes; o gerente, para cadastrar veículos, ajustar preços e acompanhar quantos carros estão disponíveis, reservados e vendidos.

→ Requisito derivado: Define os perfis de usuário e a visão-resumo por status.

3. Quais informações de cada veículo são indispensáveis?

Resposta simulada: Marca, modelo, ano de fabricação, cor, preço, quilometragem e a situação do carro (se está disponível ou não).

→ Requisito derivado: Origem dos atributos de Vehicle (RF01).

4. Como marcas e modelos se relacionam com os veículos?

Resposta simulada: Uma marca tem vários modelos, e temos vários carros do mesmo modelo — por exemplo, três Corollas de anos e cores diferentes. O tipo do carro (hatch, sedã, SUV, picape) depende do modelo.

→ Requisito derivado: Leva às entidades Brand (1:N) VehicleModel (1:N) Vehicle e ao atributo category no modelo (RF02).

5. Quais situações (status) um veículo pode ter?

Resposta simulada: Disponível, reservado (cliente deu sinal), vendido e descontinuado (saiu de linha ou foi retirado da venda).

→ Requisito derivado: Origem do enum VehicleStatus.

6. Quais buscas os vendedores mais fazem no dia a dia?

Resposta simulada: Clientes chegam pedindo coisas como “um SUV até 150 mil, de 2022 para cima”. Então buscamos por marca, modelo, tipo, faixa de preço, ano e só os disponíveis.

→ Requisito derivado: Define os filtros combináveis e a ordenação (RF03).

7. Que informações mudam depois do cadastro?

Resposta simulada: O preço muda em promoções, a quilometragem aumenta com test-drives e transferências entre lojas, e o status muda a cada reserva ou venda. A cor só muda se alguém digitou errado.

→ Requisito derivado: Define o que é editável no PATCH (RF04).

8. Existe alguma regra para a quilometragem?

Resposta simulada: Ela nunca pode diminuir. Se diminuir, foi erro de digitação ou algo pior — o sistema deveria barrar.

→ Requisito derivado: Regra de negócio implementada em Vehicle.updateMileage().

9. O que acontece com o carro depois de vendido?

Resposta simulada: Ele sai da vitrine e não pode mais voltar a ficar disponível nem ter o preço alterado, senão bagunça o histórico de vendas.

→ Requisito derivado: Status SOLD é final; veículo vendido não é editável.

10. Quando um veículo pode ser excluído do sistema?

Resposta simulada: Só quando foi vendido ou descontinuado. Um carro disponível ou reservado está fisicamente no pátio; apagá-lo seria perder o controle dele.

→ Requisito derivado: Regra de remoção (RF05) em Vehicle.ensureCanBeRemoved().

11. Quantos veículos vocês têm e qual a expectativa de velocidade?

Resposta simulada: Cerca de 300 carros hoje, com previsão de crescer. A busca precisa responder na hora, enquanto o cliente está na frente do vendedor.

→ Requisito derivado: Requisito de performance: índices, paginação e consultas sem N+1.

12. Onde o sistema será acessado e há planos de integração?

Resposta simulada: Pelo navegador nos computadores da loja. No futuro queremos mostrar o estoque no site e em portais de anúncio.

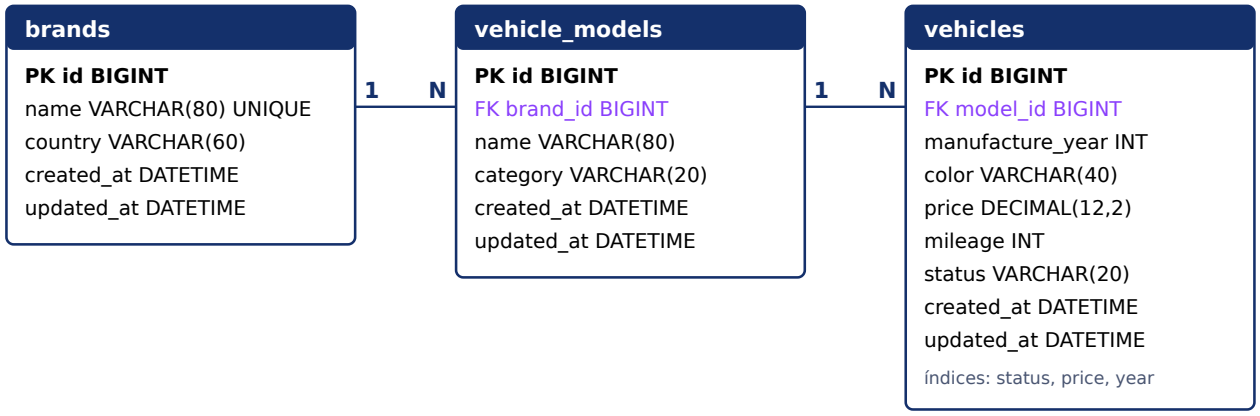
→ Requisito derivado: Justifica uma API REST documentada (Swagger) separada da interface.

3. Entidades do sistema

A partir das respostas, o domínio foi modelado em três entidades principais, uma superclasse comum e dois enumerados:

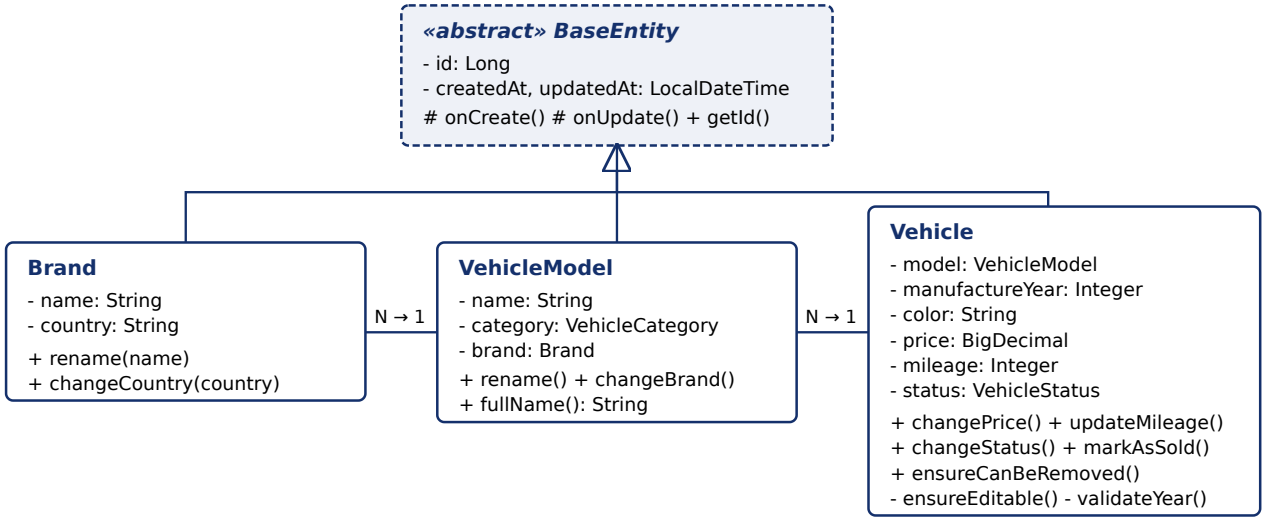
Entidade	Atributos	Descrição
BaseEntity (abstrata)	id, createdAt, updatedAt	Superclasse de todas as entidades: identificador e auditoria de criação/alteração.
Brand (Marca)	name (único), country	Fabricante do veículo. Uma marca possui vários modelos.
VehicleModel (Modelo)	name, category, brand	Modelo comercial (Corolla, Onix...), sempre vinculado a uma marca. Define o tipo do veículo.
Vehicle (Veículo)	model, manufactureYear, color, price, mileage, status	Unidade física em estoque. A marca é obtida pelo modelo (Vehicle → VehicleModel → Brand).
VehicleStatus (enum)	AVAILABLE, RESERVED, SOLD, DISCONTINUED	Situação do veículo, com as regras de transição e de remoção.
VehicleCategory (enum)	HATCH, SEDAN, SUV, PICKUP, MINIVAN, COUPE	Tipo de veículo, usado nos filtros.

Modelo entidade-relacionamento (físico)



O relacionamento é **Marca 1:N Modelo 1:N Veículo**. O veículo não guarda a marca diretamente: ela é obtida pelo modelo, o que evita inconsistências (por exemplo, um "Corolla" associado à marca errada). Os enumerados são gravados como texto (`VARCHAR`), o que torna o banco legível e protege contra mudanças na ordem dos valores.

4. Programação Orientada a Objetos



Classes e objetos

Cada conceito do negócio virou uma classe: `Brand`, `VehicleModel` e `Vehicle`. Cada carro do pátio é um **objeto** (instância) de `Vehicle`. Além das entidades, há classes com papéis técnicos bem definidos: controllers (recebem as requisições HTTP), services (casos de uso), repositories (acesso ao banco) e DTOs (formato dos dados que entram e saem da API).

Atributos

Os atributos representam o estado de cada objeto e usam tipos adequados: `BigDecimal` para preço (evitando erros de arredondamento de `double` com dinheiro), `Integer` para ano e quilometragem, enumerados para status e categoria, e referências a outros objetos para os relacionamentos (`Vehicle.model`, `VehicleModel.brand`).

Métodos

Em vez de *setters* genéricos, as entidades expõem métodos com nome de negócio: `changePrice()`, `updateMileage()`, `changeStatus()`, `markAsSold()`, `ensureCanBeRemoved()`. Cada um valida a regra antes de alterar o estado, de modo que o objeto nunca fica inválido.

Herança

- `BaseEntity` é uma classe **abstrata** com `id`, `createdAt` e `updatedAt` e com os métodos de auditoria `onCreate()` / `onUpdate()`. As três entidades a estendem (`extends BaseEntity`) e herdam esses atributos e comportamentos sem repetir código.

- As exceções seguem a mesma ideia: `BusinessRuleException` e `ResourceNotFoundException` estendem a abstrata `DomainException`. O tratador global converte cada subtipo em um status HTTP diferente (422 e 404).

Encapsulamento

Todos os atributos são `private`. O acesso externo é feito por *getters* (somente leitura) e pelos métodos de negócio. O construtor vazio exigido pelo JPA é `protected`, impedindo que outras partes do sistema criem objetos incompletos. Métodos auxiliares como `ensureEditable()` e `validateYear()` são privados: são detalhes internos da classe.

Trecho simplificado de Vehicle.java

```
public class Vehicle extends BaseEntity {          // HERANÇA

    private BigDecimal price;                         // ENCAPSULAMENTO: atributos privados,
    private Integer mileage;                          // sem setters públicos
    private VehicleStatus status;

    public void updateMileage(int newMileage) {       // MÉTODO com regra de negócio
        ensureEditable();
        if (this.mileage != null && newMileage < this.mileage) {
            throw new BusinessRuleException("A quilometragem não pode diminuir");
        }
        this.mileage = newMileage;
    }

    private void ensureEditable() { ... }              // detalhe interno escondido
}
```

Polimorfismo e abstração

O enumerado `VehicleStatus` é uma classe com comportamento próprio: cada valor responde de forma diferente a `canTransitionTo()` e `allowsRemoval()`. Os repositórios são **interfaces** (`JpaRepository`) cuja implementação é fornecida pelo Spring em tempo de execução, e o código depende apenas da abstração.

VehicleStatus.java — regras de transição

```
public enum VehicleStatus {
    AVAILABLE("Disponível"), RESERVED("Reservado"), SOLD("Vendido"), DISCONTINUED("Descontinuado");

    public boolean canTransitionTo(VehicleStatus target) {
        if (this == target) return true;
        return switch (this) {
            case AVAILABLE, RESERVED -> true;
            case DISCONTINUED -> target == AVAILABLE;
            case SOLD -> false;                        // vendido é final
        };
    }
}
```

5. Requisitos funcionais e não funcionais

Requisitos funcionais

ID	Requisito	Descrição	Endpoint
RF01	Cadastro de veículos	Permitir cadastrar veículo com modelo, marca, ano de fabricação, cor, preço, quilometragem e status. Todo veículo novo entra como Disponível.	POST /api/vehicles
RF02	Cadastro de marcas e modelos	Permitir cadastrar, editar, listar e remover marcas e modelos, associando cada modelo a uma marca e cada veículo a um modelo. Não permitir nomes duplicados nem remover marca/modelo em uso.	/api/brands, /api/models
RF03	Consulta e filtro de veículos	Permitir consultar veículos filtrando por marca, modelo, tipo, faixa de preço, faixa de ano, status e cor, com filtros combináveis, paginação e ordenação.	GET /api/vehicles
RF04	Atualização de informações	Permitir alterar preço, quilometragem, status (e cor), respeitando as regras: preço > 0, quilometragem não diminui, veículo vendido não é editado.	PATCH /api/vehicles/{id}
RF05	Remoção de veículos	Permitir remover do estoque veículos vendidos ou descontinuados; bloquear a remoção dos demais.	DELETE /api/vehicles/{id}
RF06	Resumo do estoque	Exibir a quantidade de veículos por status para acompanhamento gerencial.	GET /api/vehicles/summary

Requisitos não funcionais

ID	Categoria	Descrição
RNF01	Usabilidade	Interface web simples com filtros visíveis, formulários em janelas de diálogo e mensagens de erro claras em português (padrão Problem Details nas respostas da API).
RNF02	Performance	Consultas paginadas (máx. 100 itens por página), índices em status, preço e ano, e carregamento de modelo e marca na mesma consulta (@EntityGraph), evitando o problema N+1.

ID	Categoria	Descrição
RNF03	Integridade dos dados	Validação em três camadas: Bean Validation na entrada, regras na entidade e restrições no banco (FK, UNIQUE, CHECK de preço e quilometragem). Operações transacionais (@Transactional).
RNF04	Manutenibilidade	Arquitetura em camadas (controller, service, domain, repository), código comentado e testes automatizados de unidade e de integração.
RNF05	Portabilidade	Execução com um comando via Docker Compose (aplicação + MySQL), independente do sistema operacional.
RNF06	Interoperabilidade	API REST em JSON documentada com OpenAPI/Swagger, pronta para integração com site ou aplicativo.

Regras de negócio

- **RN01** — Preço deve ser maior que zero (armazenado com 2 casas decimais).
- **RN02** — Quilometragem não pode ser negativa nem diminuir.
- **RN03** — Ano de fabricação entre 1900 e o ano seguinte ao atual.
- **RN04** — Veículo vendido não muda de status nem pode ser editado.
- **RN05** — Descontinuado só pode voltar a Disponível.
- **RN06** — Só é possível remover veículos vendidos ou descontinuados.
- **RN07** — Não há marcas com o mesmo nome nem modelos repetidos na mesma marca; marca/modelo em uso não podem ser removidos.

6. Justificativa das tecnologias

Tecnologia	Por que foi escolhida
Java 21 (LTS)	Linguagem orientada a objetos, fortemente tipada e madura no mercado corporativo. A versão 21 tem suporte de longo prazo e recursos modernos usados no projeto, como records (DTOs imutáveis) e switch expressions.
Spring Boot 3.3	Framework mais adotado para APIs em Java. A autoconfiguração elimina boa parte da configuração manual, e o ecossistema oferece Spring MVC (REST), Spring Data JPA (persistência), Bean Validation e testes integrados.
Spring Data JPA / Hibernate	Mapeamento objeto-relacional: as classes de domínio viram tabelas, e consultas simples são geradas a partir do nome dos métodos. Specifications permitem montar os filtros dinâmicos do RF03.
MySQL 8	Banco relacional gratuito, muito difundido e adequado a dados estruturados com relacionamentos (marca → modelo → veículo). Garante integridade com chaves estrangeiras, UNIQUE e CHECK, e oferece índices para as consultas.
springdoc-openapi (Swagger UI)	Gera a documentação interativa da API automaticamente, permitindo testar cada endpoint pelo navegador.
JUnit 5, MockMvc e H2	Testes de unidade das regras de negócio e testes de integração da API com banco em memória compatível com MySQL.
Docker Compose	Sobe aplicação e banco com um único comando, garantindo o mesmo ambiente na máquina de qualquer avaliador.

A combinação **Java + Spring Boot + MySQL** é um padrão de mercado para sistemas corporativos: tem ampla documentação, comunidade grande, boa performance e separação clara de responsabilidades. O uso de JPA permite trabalhar com objetos no código enquanto o MySQL garante a integridade dos dados relacionais.